DATABASE QUERY LAB

SQL JOINs

The concept that turns separate tables into useful answers.

A JOIN combines rows from multiple tables using related columns. The JOIN type decides what happens when a match exists, and what happens when it does not.

Users		left
ID	NAME	NOTE
1	Ana	2 orders
2	Ben	none
3	Cara	none

Orders		right
ID	USER_ID	TOTAL
101	1	50
102	1	25
103	4	80

START WITH THE MATCH RULE

Every JOIN needs a reason.

The join predicate is the condition after ON.

```sql
SELECT columns
FROM Users u
JOIN Orders o
  ON u.id = o.user_id;
```

> Primary key: uniquely identifies a row in its own table.

> Foreign key: points to a row in another table.

> Cardinality: one-to-one, one-to-many, or many-to-many.

ONLY MATCHING ROWS

INNER JOIN

Returns rows that exist on both sides of the relationship.


```sql
SELECT *
FROM Users u
INNER JOIN Orders o
  ON u.id = o.user_id;
```

With the sample data, Ana appears twice. Ben, Cara, and order 103 are excluded.

PRESERVE THE LEFT TABLE

LEFT JOIN

Returns all left rows, plus matching right rows. Missing matches become `NULL`.


```sql
SELECT *
FROM Users u
LEFT JOIN Orders o
  ON u.id = o.user_id;
```

Use this when the left table is your primary list, such as all users whether or not they ordered.

OUTER JOINS KEEP UNMATCHED ROWS

RIGHT and FULL OUTER

RIGHT preserves the right table.
FULL OUTER preserves both tables.


```sql
SELECT *
FROM Users u
FULL OUTER JOIN Orders o
  ON u.id = o.user_id;
```

MySQL does not support `FULL OUTER JOIN` directly. Simulate it with left and right joins plus `UNION`.

EVERY COMBINATION

CROSS JOIN

Pairs every row from the first table with every row from the second table.

3 Users	x	4 Products	=	12 Rows

```sql
SELECT *
FROM Users
CROSS JOIN Products;
```

Use it intentionally for grids, options, calendars, sizes, products, or test data. Row counts grow fast.

A TABLE JOINS ITSELF

SELF JOIN

Useful when rows in one table point to other rows in the same table.

```sql
SELECT
  e.name,
  m.name AS manager
FROM Employees e
LEFT JOIN Employees m
  ON e.manager_id = m.id;
```

> Managers and employees

> Categories and parent categories

> Comments and replies

FIND MISSING MATCHES

Anti Join

Return rows from one table that do not have a match in another table.

```sql
SELECT u.id, u.name
FROM Users u
LEFT JOIN Orders o
  ON u.id = o.user_id
WHERE o.id IS NULL;
```

This answers questions like: which users have no orders, which products never sold, or which employees have no assignment?

FIND ROWS THAT HAVE A MATCH

Semi Join

Use `EXISTS` when you only need to know whether a match exists.

```sql
SELECT u.id, u.name
FROM Users u
WHERE EXISTS (
  SELECT 1
  FROM Orders o
  WHERE o.user_id = u.id
);
```

An inner join can duplicate Ana because she has two orders. `EXISTS` keeps each user once.

USE A BRIDGE TABLE

Many-to-many joins need a middle table.

Students	->	Enrollments	->	Classes
many classes		student_id, class_id		many students

```
SELECT s.name, c.name
FROM Students s
JOIN Enrollments e ON s.id = e.student_id
JOIN Classes c ON e.class_id = c.id;
```

Do not store comma-separated IDs in a column. Make the relationship queryable.

ON VS WHERE

Outer join filters can change the answer.

A right-table filter in WHERE can remove unmatched left rows.

```sql
SELECT *
FROM Users u
LEFT JOIN Orders o
  ON u.id = o.user_id
  AND o.status = 'paid';
```

Put optional right-side filters in ON when you still need every left row.

WHICH JOIN SHOULD YOU USE?

Pick by the rows you need.

> Only matches: **INNER JOIN**

> All primary rows: **LEFT JOIN**

> Everything from both sides: **FULL OUTER JOIN**

> Every combination: **CROSS JOIN**

> Missing matches: **anti join**

> Match exists, no duplicates: **EXISTS**